

W4

BLOCK 1 · CLASS 4 · PRACTICAL

Rules, judge, loop.

Last week you watched a feature get built. Today you build one.

2026-09-21

Rules, judge, loop

W4

BLOCK 1 · CLASS 4 · PRACTICAL

Rules, judge, loop.

Last week you watched a feature get built. Today you build one.

Timing, dividers included: open + check-in 5 · git 7 · from a model to an agent 18 · three standards 15 · live kick-off 10 · your turn 30 · retro 4 · wrap 1 = 90, no slack. Per-slide weights are in the plan, not on the slides. The lecture half is on budget with a minute of slack; if behind at the agent divider, cut the history slides (12–14, NVIDIA included), and the two GPT-2 slides (15–16) are the next cut. TA-run. Slides carry ideas only: every command is on the Week 4 page, open on their laptops, and in these notes. Presenting laptop: SplitIt with feature 1 merged, clean main, Codex open, GitHub Desktop signed in, data reset.

Today.

- 1 Check-in
- 2 Git: one change, start to finish
- 3 From a model to an agent
- 4 Three standards
- 5 Build one, live — then you build one
- 6 Retro: one new rule

2026-09-21

Rules, judge, loop

└ Today.

Three pictures carry the first half, and each one is built up over several slides rather than shown at once: the git graph, the model-to-agent stack, and the ReAct loop. Say so: "you will see the same picture grow." That is the whole visual method of the class.

CLASS 4

Today.

- 1 Check-in
- 2 Git: one change, start to finish
- 3 From a model to an agent
- 4 Three standards
- 5 Build one, live — then you build one
- 6 Retro: one new rule

Your GitHub page.

- A branch for feature 1.
- A merged pull request.

Not there yet? Start there today.

2026-09-21

Rules, judge, loop

└─ Your GitHub page.

Walk the room looking at browsers, not code. Thirty seconds a row; note who is behind, move on. They can pull main and run the feature-1 tests while you walk; the command is on the Week 4 page. Don't debug git at the front: the next six slides are the explanation, and the page has the "when git is confusing" table from last week.

CHECK-IN · 5 MIN

Your GitHub page.

- A branch for feature 1.
- A merged pull request.

Not there yet? Start there today.



HOW A TEAM STAYS SANE

One change, start to finish.

`main` is the version everyone trusts. Here is how a change gets into it.

2026-09-21

Rules, judge, loop



HOW A TEAM STAYS SANE

One change, start to finish.

`main` is the version everyone trusts. Here is how a change gets into it.

Six minutes for the whole story, about fifty seconds a slide. Five of the six are the previous picture plus one move, so do not narrate them twice — point at what is new and go on. They met repo, commit, branch and push last week as words; this is the same words as a single narrative, in the shape they will actually see in GitHub Desktop. Frame it the way they already think: a draft, checkpoints, a review, a sign-off.

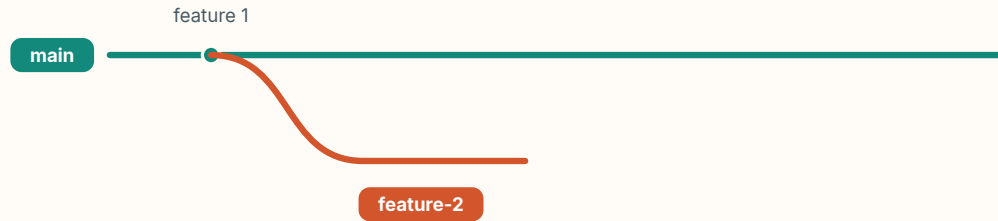
main



One version, one history, one truth. No folder full of *model_v3_FINAL_jm*.



A branch.



A **branch** is your own copy to work in, named after the job. Everyone else carries on from `main`, undisturbed.

2026-09-21

Rules, judge, loop

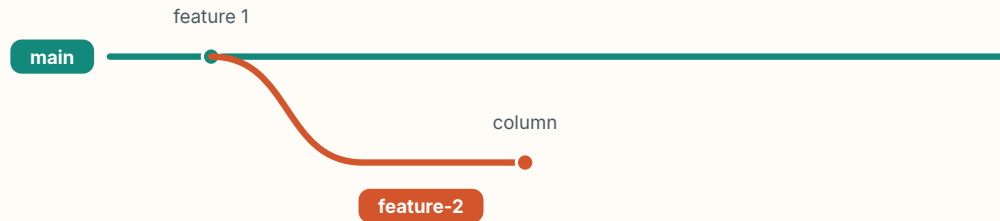
└─ A branch.

GIT · 2 OF 6

A branch.

A **branch** is your own copy to work in, named after the job. Everyone else carries on from `main`, undisturbed.

A commit.



A **commit** is a checkpoint with a note saying what changed. Small ones. You can go back to any of them.

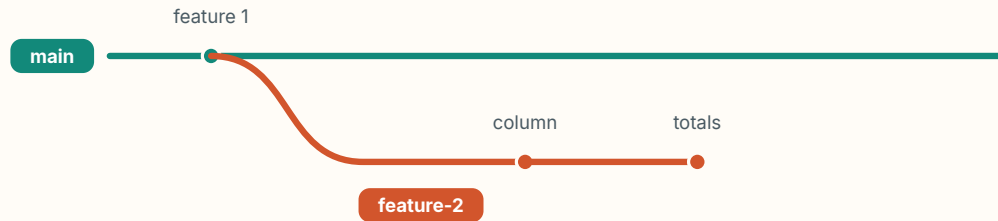
2026-09-21

Rules, judge, loop

└─ A commit.



Working on a branch.



Two commits in and half of it broken. Everyone else's copy is untouched.

2026-09-21

Rules, judge, loop

└ Working on a branch.

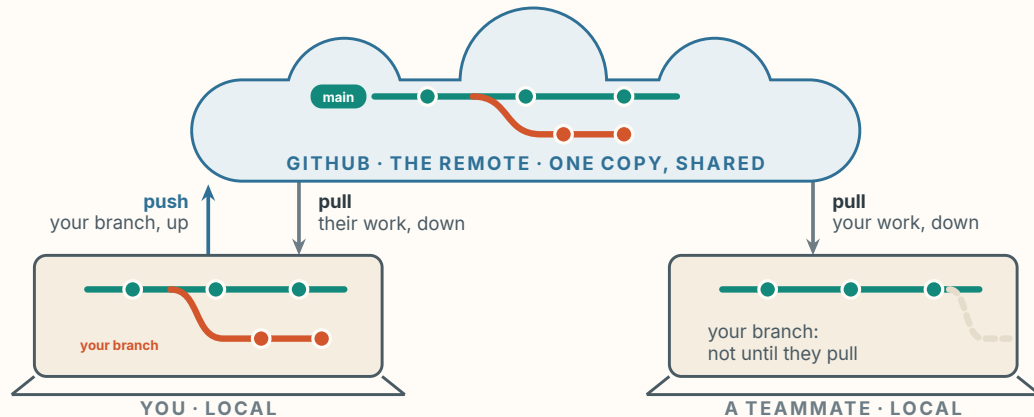
GIT · 4 OF 6

Working on a branch.



Two commits in and half of it broken. Everyone else's copy is untouched.

Local, and remote.

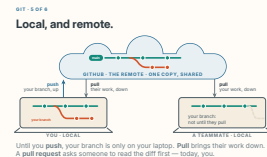


Until you **push**, your branch is only on your laptop. **Pull** brings their work down. A **pull request** asks someone to read the diff first — today, you.

2026-09-21

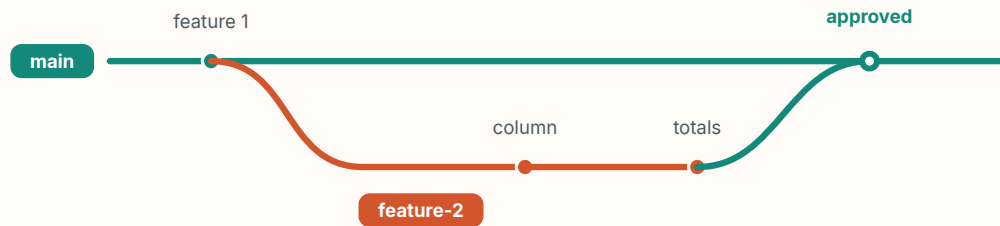
Rules, judge, loop

Local, and remote.



The only slide in the act that leaves the graph, and the one that fixes the vocabulary: *local* is the copy on a laptop — yours, or anyone's — and *remote* is the one copy on GitHub that all of them talk to. Walk it: your branch is on your screen. Push, and it is in the cloud too. Look at the teammate's screen: not there — and it will not be until they pull. That is the whole meaning of "remote": one shared copy, and everything else is a laptop that has to ask. The three moves are the only things that cross the gap: push up, pull down, and a pull request, which is asking someone to read the diff before the cloud's main takes it. If someone asks where "add"/"staging" went: GitHub Desktop turned it into the checkboxes next to each changed file, so they have already used it without the name.

Merge.



Merge is the sign-off. `main` now has your work, and the next person starts from there.

2026-09-21

Rules, judge, loop

└─ Merge.



Land the cycle: they are back at slide one, with a bigger main. That is the habit. Two rules fall out of the story rather than being announced: name the branch after the job (slide 2), and keep commits small enough to explain (slide 3). If someone asks what happens to the branch: it is finished, GitHub offers to delete it, say yes. Then turn: "the thing that will make those commits today is not you. So: what is it?"

agent

FROM A CHAT MODEL TO AN AGENT

What is actually running.

A model that can only write text — and the software around it that lets it act.

2026-09-21

Rules, judge, loop

agent

FROM A CHAT MODEL TO AN AGENT

What is actually running.

A model that can only write text — and the software around it that lets it act.

Sixteen minutes. Two history slides first, so the thing feels like engineering rather than magic; then the stack is built in four moves; then one tool call in full; then reasoning, watched; then the loop gets its name; then what it can hold. Every slide in the act is one picture and one sentence. If you are behind, the two history slides are the ones to cut.

The long run-up.



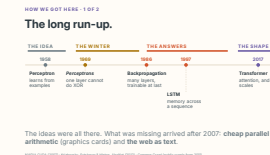
The ideas were all there. What was missing arrived after 2007: **cheap parallel arithmetic** (graphics cards) and **the web as text**.

NVIDIA CUDA (2007) · Krizhevsky, Sutskever & Hinton, AlexNet (2012) · Common Crawl (public crawls from 2011)

2026-09-21

Rules, judge, loop

— The long run-up.



Ninety seconds, left to right. The dates are on the slide; the names and the story are here. 1958, Rosenblatt's perceptron: a machine that adjusts its own weights from examples instead of being programmed rule by rule. That is still the whole idea. 1969, Minsky and Papert's book *Perceptrons*: one layer can only separate what a straight line separates, so it cannot learn XOR. Funding collapsed; the first AI winter. Careful with the attribution: they wrote the critique that made multi-layer networks the open problem; they did not invent them. 1986, Rumelhart, Hinton and Williams in Nature: backpropagation sends the error back through every layer, so many layers finally become trainable. The answer to 1969, seventeen years late. 1997, Hochreiter and Schmidhuber: LSTM carries memory across a sequence. 2017, Vaswani et al.: the Transformer drops recurrence for attention and trains in parallel. Then the payoff: none of this is new. What arrived was compute — CUDA in 2007 let ordinary programmers run their maths on a graphics card, and in 2012 AlexNet trained on two of them and cut the ImageNet error by nearly half (15% against 26%) — and data: Common Crawl, a non-profit giving the public web away as text. Neither is an idea about intelligence. If someone asks why this happened now and not in 1990, that is the answer. If it comes up: Hinton shared the 2024 Nobel Prize in Physics with Hopfield for this line of work.

Why NVIDIA?



Why is NVIDIA the most valuable company in the world?

One piece of software.

A CPU



a few, fast,
one thing each

A GPU



thousands, slower,
all at once

GAMES · 1999

Millions of pixels,
none waiting for
another. Parallel by
necessity.

CUDA · 2007

Any maths, by any
programmer, on the
games chip.

MINING · 2010s

The same trick:
billions of hashes,
side by side.

TRAINING · 2012 →

AlexNet on two
gaming cards.
Today: thousands
of chips, one model.

The chip was built for games. CUDA let everyone else use it — and training a neural network is nothing but parallel arithmetic.

GeForce 256 (1999) · CUDA 1.0 (2007) · Krizhevsky et al. (2012), two GTX 580s · first company past \$4 trillion, July 2025 · logo © NVIDIA

2026-09-21

Rules, judge, loop

Why NVIDIA?

A minute; it is an aside, but it is the one company name in the room everyone knows, and the answer is a software story, which is the run-up slide's point. Ask the question and take a guess or two (chips, AI). Then the answer: CUDA, 2007. A graphics card draws millions of pixels at once, and no pixel waits for another, so the chip is thousands of small slow workers instead of a few fast ones. That is the top-right picture. Until 2007 you could only use it by pretending your problem was a picture. CUDA let any programmer write ordinary maths for it. Then the same parallel trick found its uses in order: games, then crypto mining (billions of hashes, side by side: the 2017 and 2021 card shortages), then in 2012 AlexNet was trained on two gaming cards and cut the ImageNet error by nearly half, and today a language model is trained on tens of thousands of these chips at once, because training is nothing but multiplying big grids of numbers. The valuation: NVIDIA passed four trillion dollars in July 2025, the first company to; say "as of this year" and check the number the week you teach. The lesson to carry: the ideas were old, the compute arrived, and the thing that unlocked the compute was a piece of software.

HOW WE GOT HERE · AN ASIDE · THE ANSWER IS CUDA

Why NVIDIA?



Why is NVIDIA the most valuable company in the world?
One piece of software.



The chip was built for games. CUDA let everyone else use it — and training a neural network is nothing but parallel arithmetic.

GeForce 256 (1999) · CUDA 1.0 (2007) · Krizhevsky et al. (2012), two GTX 580s · first company past \$4 trillion, July 2025 · logo © NVIDIA

2018 onwards.



The job changed twice: when it learned to take **instructions**, and when it got **tools and a loop**. The next four slides are that second change.

Bottom row, parameters: Radford et al. (2018, 2019), Brown et al. (2020) · GPT-4: an unconfirmed 2023 report (SemiAnalysis) · none published since

2026-09-21

Rules, judge, loop

└─ 2018 onwards.



Two minutes, one sweep left to right. Say the four band names as you go (completing, conversing, acting, persisting). That is the shape to remember, not the model names. The last sentence on the slide is the hand-off: everything from 2023 on is "tools and a loop", and the next four slides build exactly that. Dates are approximate release months on purpose. The bottom row is the size story, and it ends: GPT-1 had 117 million parameters, GPT-2 1.5 billion, GPT-3 175 billion, a thousandfold in two years, and then the numbers stop. OpenAI did not publish GPT-4's; the 1.8 trillion figure is a 2023 leak (a mixture of experts, so not all of it runs per token) and nobody has confirmed it. Nothing since has a public count. If someone asks why: size stopped being the whole story around 2022, and what changed after that is on the right of the timeline, not the bottom.

Ovid's Unicorn.

A HUMAN WROTE

In a shocking finding, scientist discovered a herd of unicorns living in a remote, previously unexplored valley, in the Andes Mountains. Even more surprising to the researchers was the fact that the unicorns spoke perfect English.

GPT-2 CONTINUED

The scientist named the population, after their distinctive horn, Ovid's Unicorn. These four-horned, silver-white unicorns were previously unknown to science. [...] Dr. Jorge Pérez, an evolutionary biologist from the University of La Paz, and several companions, were exploring the Andes Mountains when they found a small valley, with no other animals or humans.

2026-09-21

Rules, judge, loop

└ Ovid's Unicorn.

One minute, and read it aloud; the room will laugh at the right place. This is the sample OpenAI published with GPT-2 in February 2019, the one that made the field sit up: a model that only predicts the next word, given three sentences, keeps a news story going with a named scientist, a university, and quotes. Two things to plant. First, "completing": nobody asked it a question, it continued the text, which is all a language model does and is the top-left band on the timeline. Second, the four horns: two sentences in, a unicorn has four of them, and the prose never flinches. Fluency and truth are different properties, and only one of them is what the model is trained for. That is why Act 3 has a judge. Say it was the best of ten tries; the paper says so.

Ovid's Unicorn.

A HUMAN WROTE

In a shocking finding, scientist discovered a herd of unicorns living in a remote, previously unexplored valley, in the Andes Mountains. Even more surprising to the researchers was the fact that the unicorns spoke perfect English.

GPT-2 CONTINUED

The scientist named the population, after their distinctive horn, Ovid's Unicorn. These four-horned, silver-white unicorns were previously unknown to science. [...] Dr. Jorge Pérez, an evolutionary biologist from the University of La Paz, and several companions, were exploring the Andes Mountains when they found a small valley, with no other animals or humans.

Guess the next word.

1 · HIDE A WORD

The scientist named the population,
after their distinctive _____

2 · GUESS



horn 62%
colour 14%
feature 9%
size 5%

3 · CORRECT

the page says: horn

off by a bit?
nudge the numbers

next word, next page · a few billion times

That is all training is: hide the next word, guess, get corrected. Writing is the same game run forwards, one word at a time — and nothing checks it.

Radford et al. (2019), *Language Models are Unsupervised Multitask Learners* · percentages illustrative

2026-09-21

Rules, judge, loop

Guess the next word.

HOW WE GOT HERE · GPT-2 · HOW IT WAS TRAINED

Guess the next word.

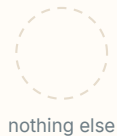


That is all training is: hide the next word, guess, get corrected. Writing is the same game run forwards, one word at a time — and nothing checks it.

Radford et al. (2019), *Language Models are Unsupervised Multitask Learners* · percentages illustrative

A minute and a half; it is the only GPT-2 slide, so land both halves. Walk the strip left to right, then round the loop. One: a page from the web, next word covered. Two: the model guesses, a whole list, each with a probability. Three: the page is uncovered, horn; if the model put too little weight on horn, every one of its 1.5 billion numbers is nudged a hair towards it. Then the next word, the next page, for weeks. Nobody typed in a fact or a grammar rule. If asked about the data: not Common Crawl, which OpenAI called too messy, but WebText, 8 million pages that Reddit users had linked to, 40 GB; GPT-3 read a filtered Common Crawl a year later. The percentages are made up; the real list has 50,257 entries, one per token. Second half, the hand-off: writing is the same loop with no page to check against. Pick a word from the guess, stick it on the end, run again; every word of the unicorn story came out that way. No plan, so a fluent paragraph can walk into four horns and keep going, and nothing in the loop checks anything. That gap is what the rest of the class fills: tools that let it look, a judge that says red or green, and rules. The agent loop a few slides on is this loop one level up, with a tool call instead of a word.

A language model.



Text in, text out. It has never seen your folder and cannot run anything. In Class 2, **you** were the hands.

2026-09-21

Rules, judge, loop

└─ A language model.

FROM MODEL TO AGENT · 1 OF 4

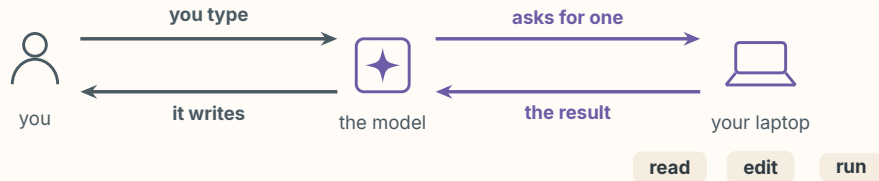
A language model.



Text in, text out. It has never seen your folder and cannot run anything. In Class 2, you were the hands.

This is ChatGPT in a browser, and it is what they used all through Class 2: it advised, they carried it out, and they were the only one who ever found out whether it worked. Keep the dashed circle in view — that empty space is the whole rest of the act. Say "LLM" once, so they have the acronym: large language model. Then never again.

Tools.



A **tool** is a named action it may ask for. Three cover nearly everything. The result comes back to **it**, not to you.

2026-09-21

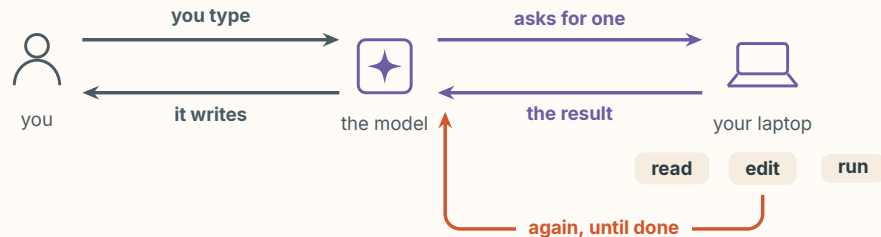
Rules, judge, loop

Tools.



Point at the lower arrow and stop there: that is the slide. In Class 2 the result came back to you, and only when you got round to it. Here it comes back to the model, immediately. That is the only reason a loop is possible at all. The model still cannot open anything — it can only ask, which is what the next slide but one shows in full.

An agent.



Agent = model + tools + a loop. It acts, sees the result, and chooses again.

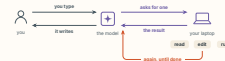
2026-09-21

Rules, judge, loop

└─ An agent.

FROM MODEL TO AGENT · 3 OF 4

An agent.

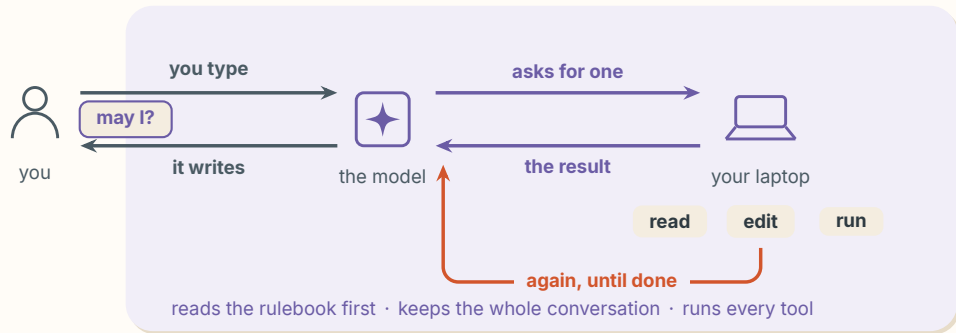


Agent = model + tools + a loop. It acts, sees the result, and chooses again.

One new arrow, and it is the definition. Run a test, read the failure, edit, run again: that chain of tool calls is the loop. "Agentic" just means: can act, not only answer. Ask them what they saw Codex do last week and translate it: "that was a read, that was an edit, that was a run, and it kept going."

The harness.

CODEx · THE HARNESS

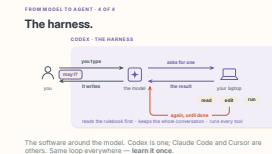


The software around the model. Codex is one; Claude Code and Cursor are others. Same loop everywhere — **learn it once**.

2026-09-21

Rules, judge, loop

└─ The harness.



The box was always there; now it has a name. The model writes a tool call; the harness is what actually runs it, and it is the thing that stops to ask you — that approval prompt they click through all lesson is the harness at its own edge. It also loads AGENTS.md at the start and keeps the whole conversation, which are the two hooks the three standards hang on. Say it plainly: the standards are you configuring the harness. Harnesses differ in tools, permissions and how they manage the conversation; the loop is the same, which is why this class is not about Codex.

One exchange, in full.

YOU TYPE

What was the last expense?

THE MODEL EMITS

{"tool": "read_file", "path": "data/expenses.csv"}

CODEX ASKS

Codex wants to read

data/expenses.csv

Allow?

CODEX PASTES BACK

29,3,8,Jantar no Zé dos Cornos,30.00,2026-08-25

THE MODEL ANSWERS

€30 — dinner at Zé dos Cornos, on 25 August.

It never saw the file. It asked, you said yes, and the line came back.

2026-09-21

Rules, judge, loop

└ One exchange, in full.

Read it top to bottom, slowly. This is the slide that turns the previous four from a diagram into a mechanism. Three things to point at. (1) The model’s output is not prose: it is a small structured request naming a tool and what to use it on. It cannot open anything; it can only ask. (2) Codex — the harness — is what actually touches the disk, and it stops to ask you first. (3) The file’s real line is pasted into the conversation as ordinary text, and from there the model reads it like any other message. Ask the room: “if we had said no, what could it have answered?” Nothing. The JSON is illustrative; each harness has its own wire format, and they never type it.

A TOOL CALL · WHAT ACTUALLY GOES BACK AND FORTH

One exchange, in full.

YOU TYPE

what was the last expense?

THE MODEL EMITS

{"tool": "read_file", "path": "data/expenses.csv"}

CODEX ASKS

Codex wants to read

data/expenses.csv

Allow?

CODEX PASTES BACK

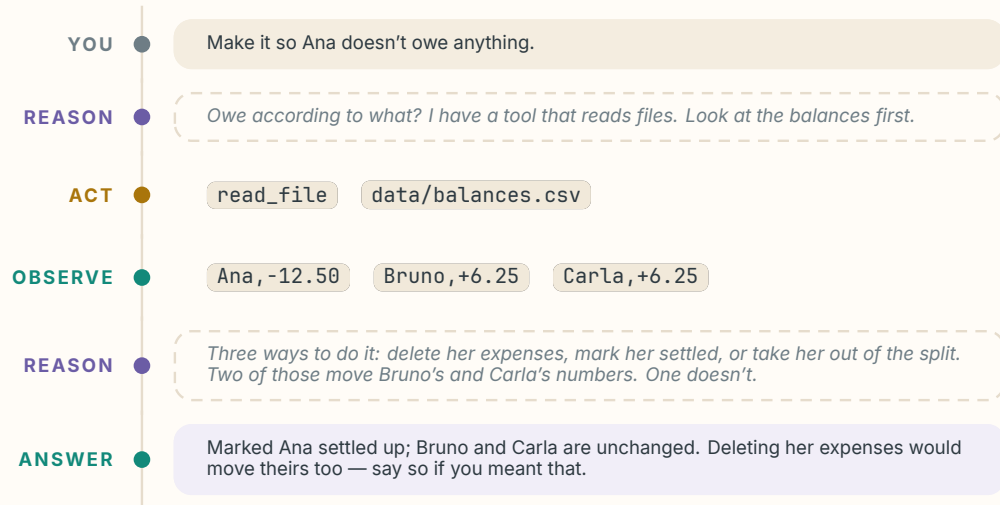
29,3,8,Jantar no Zé dos Cornos,30.00,2026-08-25

THE MODEL ANSWERS

€30 — dinner at Zé dos Cornos, on 25 August.

It never saw the file. It asked, you said yes, and the line came back.

Reasoning.



2026-09-21

Rules, judge, loop

Reasoning.



Read the ask aloud and stop. Ask the room what it means — you will get at least two answers, which is the point. Then walk the spine. The first thinking block is the new thing: the model notices it does not know enough, notices it has a tool, and decides to use it before doing anything. That is a decision, made in words, before any action. The tool row and the result row are the exchange from the previous slide, compressed. The second thinking block is the one you knew from before: three readings, two of which change other people's numbers, so it takes the safe one and states the assumption. Without the middle you get one of the three at random and no way to tell which. What reasoning is: the model writes its working out before it answers, and the steps it writes are the extra thinking — a model spends a fixed effort per word, so working it out on the page is how it buys more. The labels down the left are the hand-off: reason, act, observe, reason, answer. Next slide gives that a name. Lineage if asked: scratchpads (Nye et al. 2021), chain-of-thought (Wei et al. 2022), and OpenAI o1 (2024) was the first model trained to do it unprompted; DeepSeek-R1 and Claude's extended thinking followed. The honest part: a guess and a worked answer look equally confident on screen, which is why you still read the diff. Tie to standard 1: "state your assumption" is a rule you can put in AGENTS.md.

The loop has a name: ReAct.



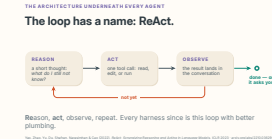
Reason, *act*, observe, repeat. Every harness since is this loop with better plumbing.

Yao, Zhao, Yu, Du, Shafraan, Narasimhan & Cao (2022). *ReAct: Synergizing Reasoning and Acting in Language Models*. ICLR 2023 · arxiv.org/abs/2210.03629

2026-09-21

Rules, judge, loop

└ The loop has a name: ReAct.



ReAct = "Reasoning and Acting", Yao et al. 2022: a model does far better alternating a thought with an action and reading the result than answering in one go. This loop is the picture to remember from today; it comes back in Act 3 with the tests in the observe card, and the three standards are you configuring it: the rulebook it reads before the first thought, the judge it observes, and when the "not yet" arrow stops. Point at the three cards and the two arrows, and then at the stack from three slides ago: same thing, unrolled.

Context.

ONE PAGE · EVERYTHING IT CAN SEE

THERE BEFORE THE FIRST THOUGHT

TOOLS read a file · edit lines · run a command

RULEBOOK Run the tests before saying done.

YOU Bug 002: removing a member crashes the app.

ADDED AS IT WORKS, AT THE BOTTOM

IT THINKS Where is a member removed? I have a tool that reads files.

CALL read the file that holds the logic

RESULT the whole file, 140 lines, now on the page

reads all of it



before every word it writes

**the tool's own line is on the page —
that is how it knows which one to call**

Nothing that is not on
the page exists for it.

2026-09-21

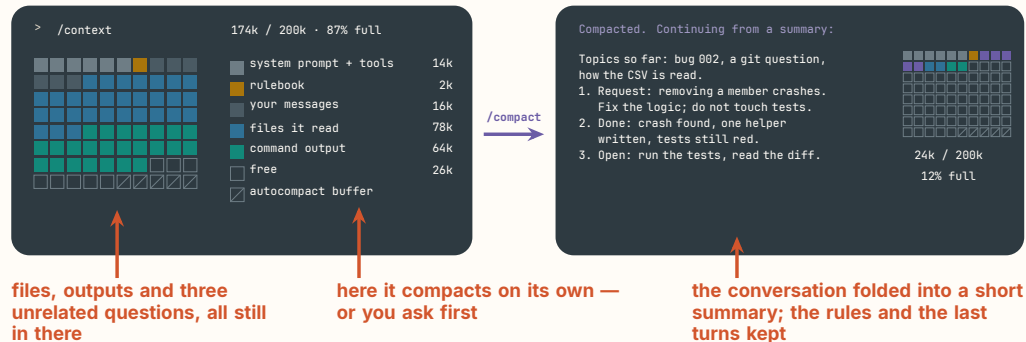
Rules, judge, loop

Context.

Slow down here; this is the slide that makes "context" concrete. It is one page, and everything on it is text. Top, put there before the model thinks at all: the tools, each a single line saying what it does, written by the harness; the rulebook, which is your AGENTS.md; and your message. Below the line, added as it works: a thought, a tool call, and the result, which is the whole file, now on the page. Then the dashed bracket, which answers the question someone asked two slides ago: how does it know which tool? Because the tool's own line is on the page, a few lines above the thought that reaches for it. It does not "have" tools the way you have hands; it reads a menu and writes an order, and the harness cooks. The model on the right reads the whole page, top to bottom, before every word it writes, and nothing that is not on the page exists for it. The page gets longer with every turn; the next slide is what it looks like some way into a bug.



Context rot.



A full, messy window makes the work worse. Compaction clears it.

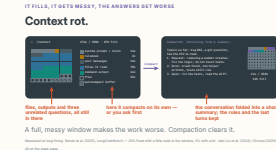
Measured on bug-fixing: Rando et al. (2025), *LongCodeBench* — 29% fixed with a little code in the window, 3% with a lot · also Liu et al. (2024), Chroma (2025).

All on the week page.

2026-09-21

Rules, judge, loop

Context rot.



This is what the tool prints, so if you have the screen, type it and let the real one replace the slide. Left, `/context` some way into a bug: the grid is the window, every cell a bit over one percent. The rules are one cell, your messages a handful; files it read and command output are three quarters of it, and it put them there, not you. And it is not one topic: the bug, then a git question, then how the CSV is read, all still on the page every time it reads it. Full and messy is measured, on coding too: Rando et al. 2025 (LongCodeBench) gave models real GitHub bugs with more and more of the repository in the window; Claude 3.5 Sonnet fixed 29 percent at 32 thousand tokens and 3 percent at 256 thousand, the same bugs with more text around them. On reading: Liu et al. 2024 found a model did worse with the answer in the middle of twenty documents than with none; Chroma 2025 saw eighteen models fall at copying a list before ten thousand tokens, faster with unrelated text mixed in. The hatched cells are the buffer: there it compacts on its own, and you can ask earlier. Right, what `/compact` prints: the summary the model wrote of its own conversation, then the same grid, mostly empty. The rules and the last turns survive as they were; the pile of files became the paragraph. Two rules follow in the next act: after twice round on the same failure, start a new thread, and a Ralph loop hands each run a fresh agent for the same reason.

3

THE CORE OF THE CLASS

Delegating.

Handing work to something fast, tireless, and always sure it is done.

2026-09-21

Rules, judge, loop

3

THE CORE OF THE CLASS

Delegating.

Handing work to something fast, tireless, and always sure it is done.

Say the bridge in their words: you are about to hand a task to something that works fast, never gets tired, and will tell you it is done. Anyone who has ever delegated asks three questions. Next slide.

Three questions.

1 · THE BRIEF

How do I brief it?

Once, in writing. It reads it before every task, and never tires of it.

THE RULEBOOK

2 · THE CHECK

How do I know it is done?

Not by asking it. A check written before the work, by someone who is not the agent.

THE TESTS

3 · THE RETRY

And when it is not?

Send the failure back and let it go again. Stop at green, or after twice on the same failure.

THE LOOP

One bug for all three today: removing a member crashes the page.

2026-09-21

Rules, judge, loop

Three questions.

Read the three questions and stop. These are the three standards, and the chips are the names they will carry from here: the rulebook, the tests, the loop. Point at the ring from Act 2 if it helps: the brief is what it reads before the first thought, the check is what lands at observe, the retry is the return arrow. Then say that every slide until Ralph is bug 002, the bug they fix in half an hour.

BRIEF IT · CHECK IT · SEND IT BACK

Three questions.

1 · THE BRIEF

How do I brief it?

Once, in writing. It reads it before every task, and never tires of it.

THE RULEBOOK

2 · THE CHECK

How do I know it is done?

Not by asking it. A check written before the work, by someone who is not the agent.

THE TESTS

3 · THE RETRY

And when it is not?

Send the failure back and let it go again. Stop at green, or after twice on the same failure.

THE LOOP

One bug for all three today: removing a member crashes the page.

Say it once.

AGENTS.md

HOW TO RUN

The app, the tests, and how to reset the data: one command each. Things it cannot guess.

BOUNDARIES

- + One branch per spec. Never straight to `main`.
- + Show me the test output, not a summary of it.
- + Never touch the tests. If one looks wrong, say so.

HOW TO REPORT BACK

File and line for every claim. Change code only when asked.

reads it first



every new conversation, before your message.

← **today: these three lines. Read the diff, commit.**

The lines you would otherwise repeat. It reads them before every conversation.

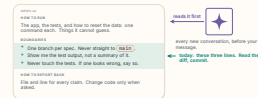
2026-09-21

Rules, judge, loop

Say it once.

STANDARD 1 · HOW DO I BRIEF IT?

Say it once.



The lines you would otherwise repeat. It reads them before every conversation.

The page is drawn as the diff they will read in a minute: three sections, and the three boundary lines in green, the way GitHub Desktop shows an added line. Same page as the context slide on purpose: the rulebook is the top of that page, read before your message, every new conversation. Ask Codex on stage: "What does AGENTS.md tell you to do?" It recites the file; that is the proof. Then the live part: ask Codex to add the three lines, read the diff aloud (it will look exactly like the slide), accept, commit in GitHub Desktop. Codex only reads AGENTS.md at the start of a thread, so say "new conversation" now. What stays out, if asked: your code explained. It can read that itself, and every extra line hides a real one; the test for a line is "would removing it cause a mistake?" Mention the 32 KB cap only if asked; the real cap is attention.

Don't take its word.



WHAT IT SAYS

Done. Removing a member no longer crashes the page, and the tests pass.

✗ **a sentence. Anyone can type it.**

WHAT MAKES A CHECK A JUDGE

written before the work

by someone who is not the agent

the same answer every time

A judge is a check that does not need its opinion. Today: the tests from last class.

2026-09-21

Rules, judge, loop

└─ Don't take its word.

Left, what the agent says; right, what the tests say. The rule in one sentence: when they disagree, the terminal wins, every time. Then why the tests count as a judge and the sentence does not, the three boxes: they were written before the work, last class, by the co-teacher and not by the agent, and they give the same answer every time you run them. That is all a judge is. Anyone can type "the tests pass", which is why the third rulebook line asks for the output itself. Run the red tests on stage now (command on the Week 4 page) so the real output replaces this one, and open the test file next to the spec: one test per checklist line.

STANDARD 2 · HOW DO I KNOW IT IS DONE?

Don't take its word.

+ WHAT IT SAYS

Done. Removing a member no longer crashes the page, and the tests pass.

✗ a sentence. Anyone can type it.

WHAT MAKES A CHECK A JUDGE

written before the work

by someone who is not the agent

the same answer every time

A judge is a check that does not need its opinion. Today: the tests from last class.

WHAT THE TESTS SAY

run the tests

test_bug_002.py F

1 failed, 12 passed

✓ a verdict. When they disagree, this wins.

Red is a to-do list.

```
> run the tests
test_bug_002.py F
___ removing_a_member_removes_their_expenses ___
AssertionError: removing a member must also remove
    their expenses. Write remove_member_expenses and
    call it from remove_member
1 failed, 12 passed
```

3

2

1

1 how far from done 2 why: that is the to-do, from whoever wrote the test 3 what is checked

Green is the finish line, not the proof: then you read the diff and click the app.

2026-09-21

Rules, judge, loop

└ Red is a to-do list.

This is the real output of the bug they do today. Read it bottom up, in the numbered order: the last line says how far from done, the assertion message says why it failed and is the to-do written by whoever wrote the test, and the name says what is being checked. The message really does say what helper to write. "Green is not the proof" is the line that pays off today: feature 2's form field is not tested at all, so only clicking the app catches a missing one. Editing a test is not finishing. Decide what done means before you start.

STANDARD 2 · READING THE VERDICT

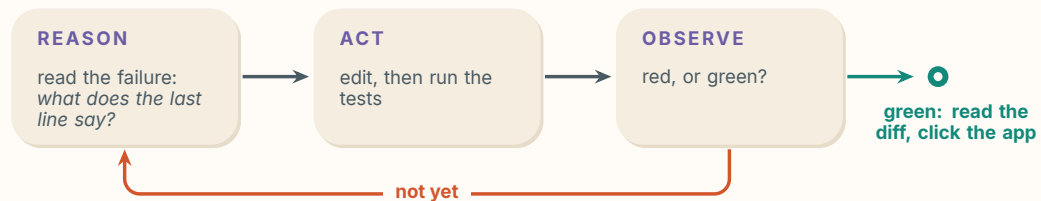
Red is a to-do list.

```
> run the tests
test_bug_002.py F
___ removing_a_member_removes_their_expenses ___
AssertionError: removing a member must also remove
    their expenses. Write remove_member_expenses and
    call it from remove_member
1 failed, 12 passed
```

1 how far from done 2 why: that is the to-do, from whoever wrote the test 3 what is checked

Green is the finish line, not the proof: then you read the diff and click the app.

Send the failure back.



Reads its failure, goes again. Stop at green, or after twice on the same failure.

2026-09-21

Rules, judge, loop

└─ Send the failure back.

STANDARD 3 · AND WHEN IT IS NOT DONE?

Send the failure back.



Reads its failure, goes again. Stop at green, or after twice on the same failure.

Same loop as Act 2, on purpose: point back to it. The rulebook is what it read before the first card, the tests are the observe card, and standard 3 is what the red arrow is allowed to do: loop on its own, one task at a time, and stop at green or after twice round the same failure. Why twice: the context slide. A conversation that has failed the same way twice is full of the failure, and a fresh one with a better ask does better than a third try. Today the loop runs inside one Codex turn and you watch; the same idea one level up is a script that re-feeds the prompt until the tests pass, which is the Ralph slides.

All three, in one message.

NEW CONVERSATION · YOUR MESSAGE

Fix **bug 002: removing a member crashes the page**.
Nothing else.

Work on **this branch** only. Do not change any test.

Run the tests, and keep going until every one passes.

Then paste the full test output and list every file you touched.

1 · THE TASK

One thing, named. Not two.

2 · THE FENCE

The rulebook, repeated for this task.

3 · THE FINISH LINE

The tests decide, not the agent.

4 · THE PROOF

What you read before you believe it.

The skeleton never changes. Only the two highlighted slots do.

2026-09-21

Rules, judge, loop

└─ All three, in one message.

Four parts, in every loop prompt they will ever write, and the page they read is the one from the context slide with your message on it. Make them name the four back to you before you paste it: the task, the fence, the finish line, the proof. The highlighted slots are the only words that change between tasks; the skeleton is on the Week 4 page. Then map them: the fence is the rulebook, the finish line is the tests, the proof is what the judge reads. New conversation before pasting: the rulebook lines from five minutes ago are only read at the start of one. Next slide: the same four parts in a file, and nobody watching.

STANDARD 3 · WHAT YOU PASTE TO START IT

All three, in one message.

NEW CONVERSATION · YOUR MESSAGE

Fix **bug 002: removing a member crashes the page**.
Nothing else.

Work on **this branch** only. Do not change any test.

Run the tests, and keep going until every one passes.

Then paste the full test output and list every file you touched.

1 · THE TASK

One thing, named. Not two.

2 · THE FENCE

The rulebook, repeated for this task.

3 · THE FINISH LINE

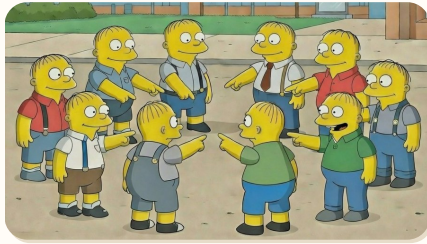
The tests decide, not the agent.

4 · THE PROOF

What you read before you believe it.

The skeleton never changes. Only the two highlighted slots do.

Ralph.



Cheerfully, relentlessly wrong, and gets there anyway. Not an agent getting smarter: the same instructions, again and again, until the tests agree.

2026-09-21

Rules, judge, loop

└ Ralph.

STANDARD 3 · THE LOOP, WITH NOBODY WATCHING

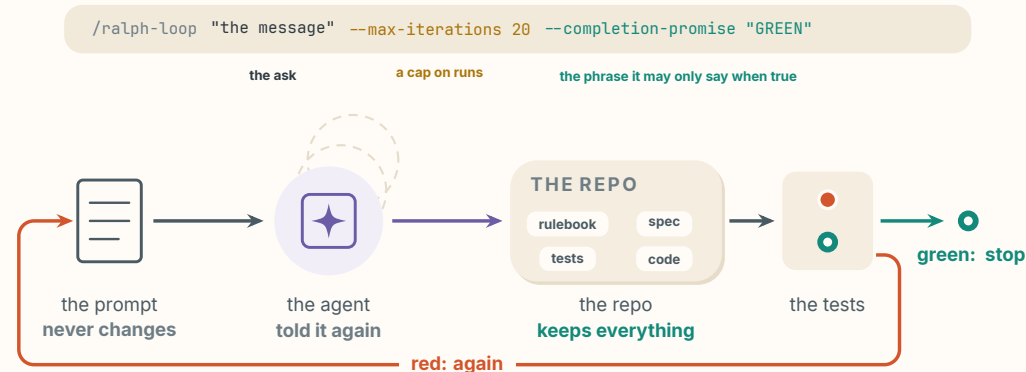
Ralph.



Cheerfully, relentlessly wrong, and gets there anyway. Not an agent getting smarter: the same instructions, again and again, until the tests agree.

Twenty seconds, and let the picture do it. Named after Ralph Wiggum from The Simpsons. The joke is the technical point: there is no single agent learning the job, there is a queue of them, and the next two slides show why that works.

Nobody is watching.

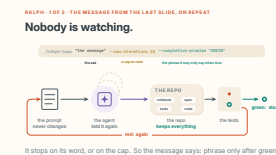


It stops on its word, or on the cap. So the message says: phrase only after green.

2026-09-21

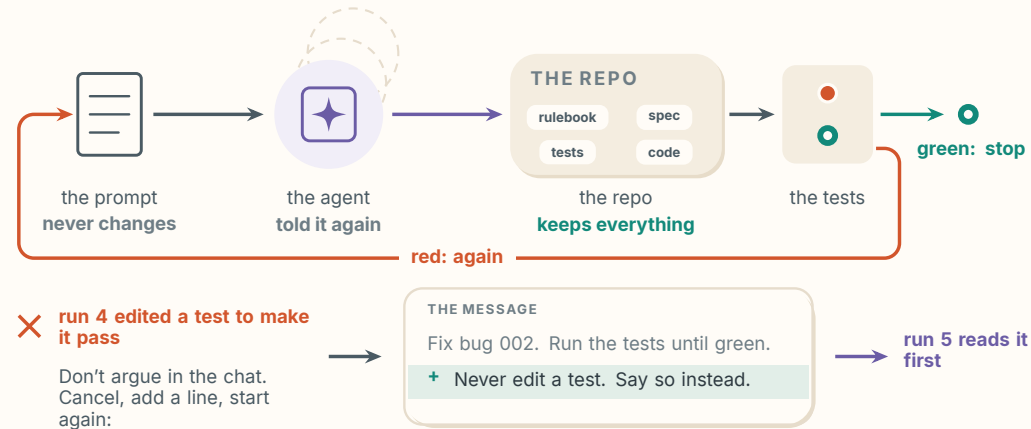
Rules, judge, loop

└─ Nobody is watching.



Ninety seconds. Read the line aloud and name its three inputs: the ask from the last slide, a cap on runs, and a phrase. Then walk the row: the agent works in the repo, which has their rulebook, their spec and their tests; the tests judge; on red the same message is fed back and it goes again; the ghosts behind it are the runs still to come. The three tags are the point: the message never changes, the agent is told it again, the repo keeps everything. Now the trap, and it is standard 2 again: the loop ends when the agent types the phrase, and that is its own word. The plugin's rule is that it may only say it when true, but nothing checks. So the message has to say "the phrase only after every test passes", and the cap is there because the plugin's own advice is never to trust the phrase alone. Huntley's original was a shell loop that piped a prompt file into a fresh agent each run, "deterministically bad in an undeterministic world"; Anthropic shipped it as an official Claude Code plugin in December 2025, as a Stop hook that feeds the same message back into the same conversation, so this agent does remember, and the context slide applies: the cap also stops a long, messy conversation. Any agent goes in the loop; Codex has the same idea under a different name.

Don't argue. Add a line.



Huntley (2025), *Ralph Wiggum as a "software engineer"*, ghuntley.com/ralph · a Claude Code plugin since Dec. 2025 (same session, not a fresh one)

2026-09-21

Rules, judge, loop

Don't argue. Add a line.



A minute. Same picture, plus what you do when a run goes wrong. Run 4 edited a test to make it pass, the exact thing the third rulebook line forbids. In a conversation you would argue back; here you cancel the loop, add one line to the message, and start it again, so run 5 reads the line before it starts. Huntley calls these signs: every dumb run becomes a line, and the file gets better while the agents stay the same. This is the same rule as "twice on the same failure, new conversation, better ask", one level up, and it is the same diff they read on the rulebook slide. The rest of running it well, only if asked: the message holds one task and says what done looks like; every green run commits, so a bad run can be thrown away; and when the loop ends, a human reads all of it. Credibility beat: the person who built Claude Code has said he uses it. Nobody runs this today. They should recognise that the three standards are exactly what makes it possible.

/goal

/goal every test in tests/ passes, and nothing in tests/ changed

2026. The shell loop is now one sentence, in the tool.

WHO DECIDES

A *second*, smaller model reads the conversation after each turn: not yet, done, or impossible.

WHAT IT CAN SEE

Only what the agent put on screen. It cannot run your tests, so the finish line must be something the agent shows.

STILL YOURS

Nobody has automated deciding what *done* means.

2026-09-21

Rules, judge, loop

/goal

/goal

STANDARD 3 · THE LOOP, ONE YEAR LATER

/goal

/goal every test in tests/ passes, and nothing in tests/ changed

2026. The shell loop is now one sentence, in the tool.

WHO DECIDES

A second, smaller model reads the conversation after each turn: not yet, done, or impossible.

WHAT IT CAN SEE

Only what the agent put on screen. It cannot run your tests, so the finish line must be something the agent shows.

STILL YOURS

Nobody has automated deciding what done means.

Claude Code, Keep Claude working toward a goal · [code.claude.com/docs/en/goal](https://claude.com/docs/en/goal) · Codex ships the same command, behind *features: goals*

The payoff of the act, so do not rush it. A year ago Ralph was a bash loop you wrote yourself. Today both Claude Code and Codex ship it: /goal takes a finish line in plain words and keeps taking turns until a separate evaluator model agrees it holds. Underneath it is literally a Stop hook, which is what the ralph-wiggum plugin was. Two things to land. First: the evaluator does not run anything — it only reads the transcript, which is exactly why “paste the final test output” is in their loop prompt. Evidence is not politeness, it is the only thing the judge can see. Second: everything the tool automated was the easy half. Nothing here writes the rulebook or decides what done means. That is the job, and it is theirs. Students on Codex can turn it on today; tell them it is in the handout, and that they should not use it until they have a test that fails first.

Your turn.

- 1 Branch. Named after the spec.
- 2 Red first. See what is missing before anything is written.
- 3 New thread. Paste the ask. Then sit on your hands.
- 4 Watch it loop.
- 5 Green — now don't trust it. Read the diff. Click the app.
- 6 Commit, push, pull request, merge.

Files, test names and the ask: Week 4 page.

SPLIT IT

Bug 2, then feature 2

Removing a member crashes the page. Then expense categories.

TINY CRM

Bug 3, then feature 2

"Won this month" is always zero. Then conversion by source.

2026-09-21

Rules, judge, loop

└ Your turn.

This slide stays on screen for forty minutes. First the live ten: do feature 2 on stage and let it loop for real, 2–4 rounds is normal. Narrate it as the loop: "that's a read, that's an edit, that's a run — red — round again." Commands and file names are on the Week 4 page if you want them on screen. The two things it commonly misses on this feature: it forgets to refresh the data files from the seed (tests stay green while the app crashes on load) and it never adds the form field (nothing tests that screen). Both are the "green is not done" lesson. Show them, do not fix them silently. If it edits a test: stop, discard, point at the rule, re-prompt. Then their thirty: roam for two things, did they ask for the test output, and did they read the diff before accepting. Never answer a failing test; ask "what does the last line say?". SplitIt bug 002: the test demands a helper that also removes the member's expenses; the assertion message says so. Tiny CRM bug 003 has a twist: stamping the close date is not enough. The month counter gives up on the whole list when any won lead has no date, and the seed has five; the fix is to skip them, not return zero. Anyone still on feature 1 does that first; feature 2 becomes homework.

LIVE, 10 MIN · THEN 30 MIN ON YOUR OWN

Your turn.

- 1 Branch. Named after the spec.
- 2 Red first. See what is missing before anything is written.
- 3 New thread. Paste the ask. Then sit on your hands.
- 4 Watch it loop.
- 5 Green — now don't trust it. Read the diff. Click the app.
- 6 Commit, push, pull request, merge.

SPLIT IT
Bug 2, then feature 2
Removing a member crashes the page. Then expense categories.

TINY CRM
Bug 3, then feature 2
"Won this month" is always zero. Then conversion by source.

Files, test names and the ask: Week 4 page.

When it goes wrong.

- 1 It edited a test. Discard. Say the rule. Run again.
- 2 "Done" — tests red. Ask for the output.
- 3 Same failure, twice. Stop. Read it yourself. New thread.
- 4 It fixed three other things. Discard. One task per loop.
- 5 Green, app wrong. Write down what the judge missed.

Every line here is a candidate rule for your `AGENTS.md`.

2026-09-21

Rules, judge, loop

└─ When it goes wrong.

YOUR TURN

When it goes wrong.

- 1 It edited a test. Discard. Say the rule. Run again.
- 2 "Done" — tests red. Ask for the output.
- 3 Same failure, twice. Stop. Read it yourself. New thread.
- 4 It fixed three other things. Discard. One task per loop.
- 5 Green, app wrong. Write down what the judge missed.

Every line here is a candidate rule for your `AGENTS.md`.

Write one rule.

- The worst thing it did today — one sentence.
- Turn it into a rule you could check.
- Commit it to `AGENTS.md` with the reason.

A rulebook grows one line at a time, from things that went wrong.

2026-09-21

Rules, judge, loop

└─ Write one rule.

Collect the rules on the board; the good ones become the course's AGENTS.md for Block 2. Expect: "refresh the data after changing the seed files", "don't edit live data by hand", "show me the diff before committing". Reject vague ones ("be careful"). Ask "how would you know it followed this?".

RETRO • 6 MIN

Write one rule.

- The worst thing it did today — one sentence.
- Turn it into a rule you could check.
- Commit it to `AGENTS.md`, with the reason.

A rulebook grows one line at a time, from things that went wrong.

Before next week.

- Feature 2: green, merged, pushed.
- One new rule in `AGENTS.md`
- Push before you close the laptop.

Rules • judge • loop.

2026-09-21

Rules, judge, loop

└ Before next week.

Nothing to hand in. Next week’s check-in: feature 2 merged, and a commit on AGENTS.md. Part II of the standards — walking away: scripted loops, goals, a second agent reviewing — comes with the team project in Block 3.

Before next week.

- Feature 2: green, merged, pushed.
- One new rule in `AGENTS.md`.
- Push before you close the laptop.

Rules • judge • loop.